LSE – MISDI 2026

AI Code Camp. Practical Skills for Research Professionals

June, 2026

Dr. Cody Dodd
LSE | Service Canada | ResearchAI

Intros and Warm-Up (10 min) — “Ground setting”

- Introduce yourself, and your interest in AI or key learning in AI you hope to gain

What this course is about:

A hands-on, modern introduction to AI systems, from classic deterministic models to generative transformers, and how to combine them into real, reliable solutions.

Over the next 3 days, you will:

- Build and run Python scripts

- Compare deterministic and generative AI

- Explore how LLMs behave (and misbehave)

- Learn how agentic systems orchestrate tools

- Create your own AI project for the final showcase

This is not a programming course, nor is it a theory course.
You will experiment, break things, debug, and build, while picking up critical skills.

Learning outcomes

By the end of this camp, you will be able to:

- **Configure** AI through Python scripts (the foundation of deterministic logic)

- **Understand** how scripts translate into agentic workflows (LLMs orchestrating tools, not guessing)

- **Explain** why Python still matters (grounding, reproducibility, auditability)

- **Describe** the difference between traditional AI and LLMs using concepts like deterministic logic and transformers

- **Compare** deterministic vs generative systems and know when to use each

- **Identify** sources of AI risk including bias, privacy risks, hallucinations, sycophancy, pollution, and non-determinism

- **Build** and present an AI project scored on interesting, practical, and mindful

Introducing Dr Cody Dodd - Relevant work

AI governance researcher, practitioner, and educator. Works with governments, NGOs, and enterprises on validation-first AI, citizen insight, and automation architectures.

Focus areas:

- Deterministic vs generative systems

- AI governance & risk

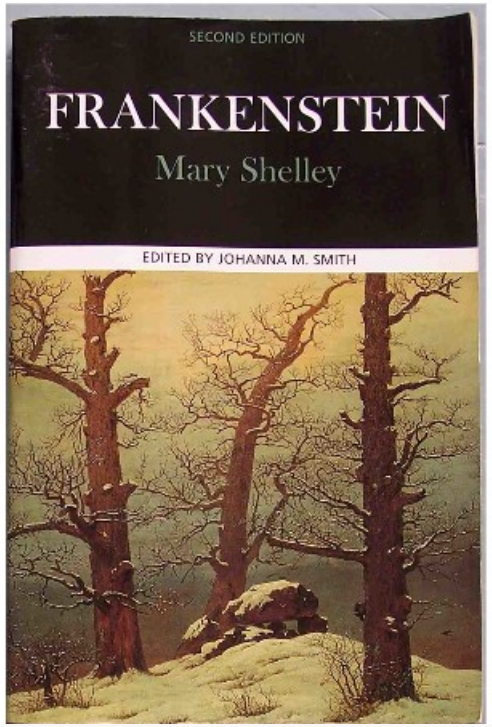
- Hybrid/agentic architectures

- Public-sector and research AI enablement

My goal: Help you understand how AI actually works, and how to build systems you can trust.

Introducing Dr Cody Dodd - Relevant work

Favourite “AI” book



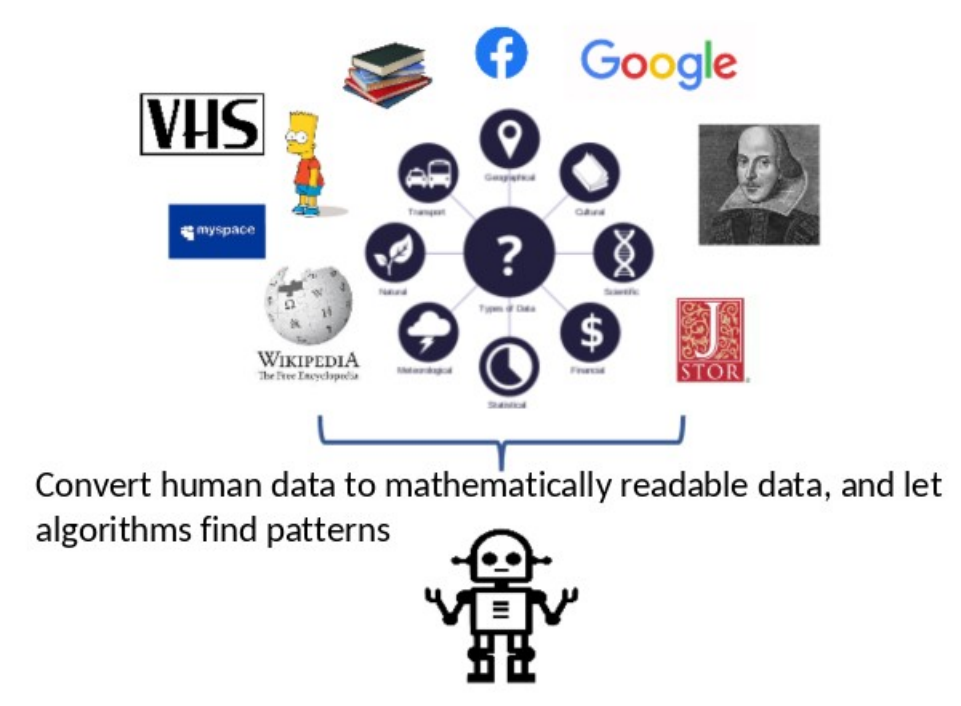
Introducing Dr Cody Dodd - Relevant work

For centuries, we have been fascinated with human-like machines - Bodies without souls. The “golem”, unfinished, incomplete, mechanical, without emotion, without learning.

Introducing Dr Cody Dodd - Relevant work

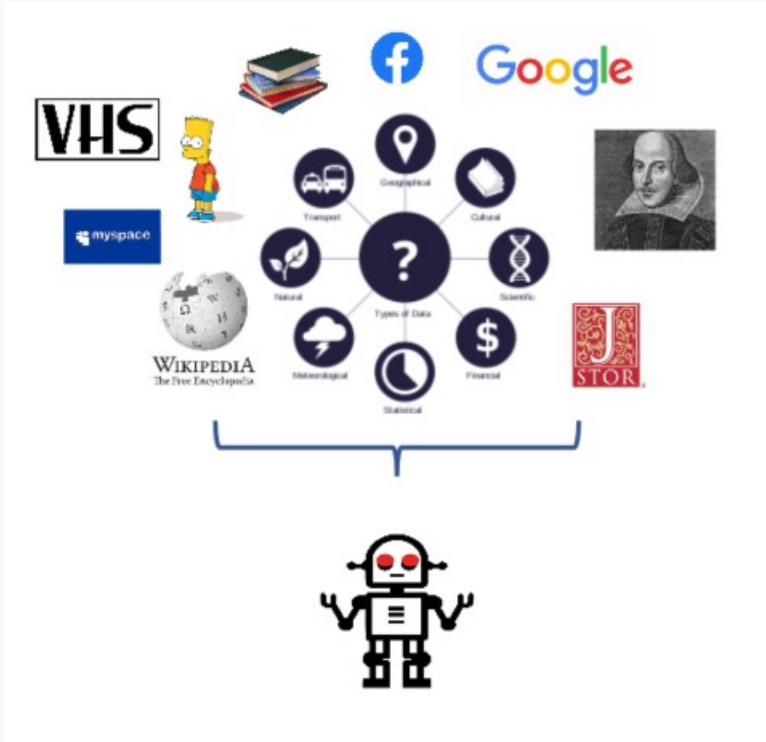
But what if we could get them to learn from our social data?
- Support text analysis
- Match employee aspirations to professional development
- Identify outliers in serviced experiences
- Optimize agricultural yields

... by robustly finding patterns



Introducing Dr Cody Dodd - Relevant work

But this can:
- Magnify biases
- Create self-fulfilling prophecies
- Misinterpret noise
- Expose privacy risks



I’m afraid I cannot do that, Dave

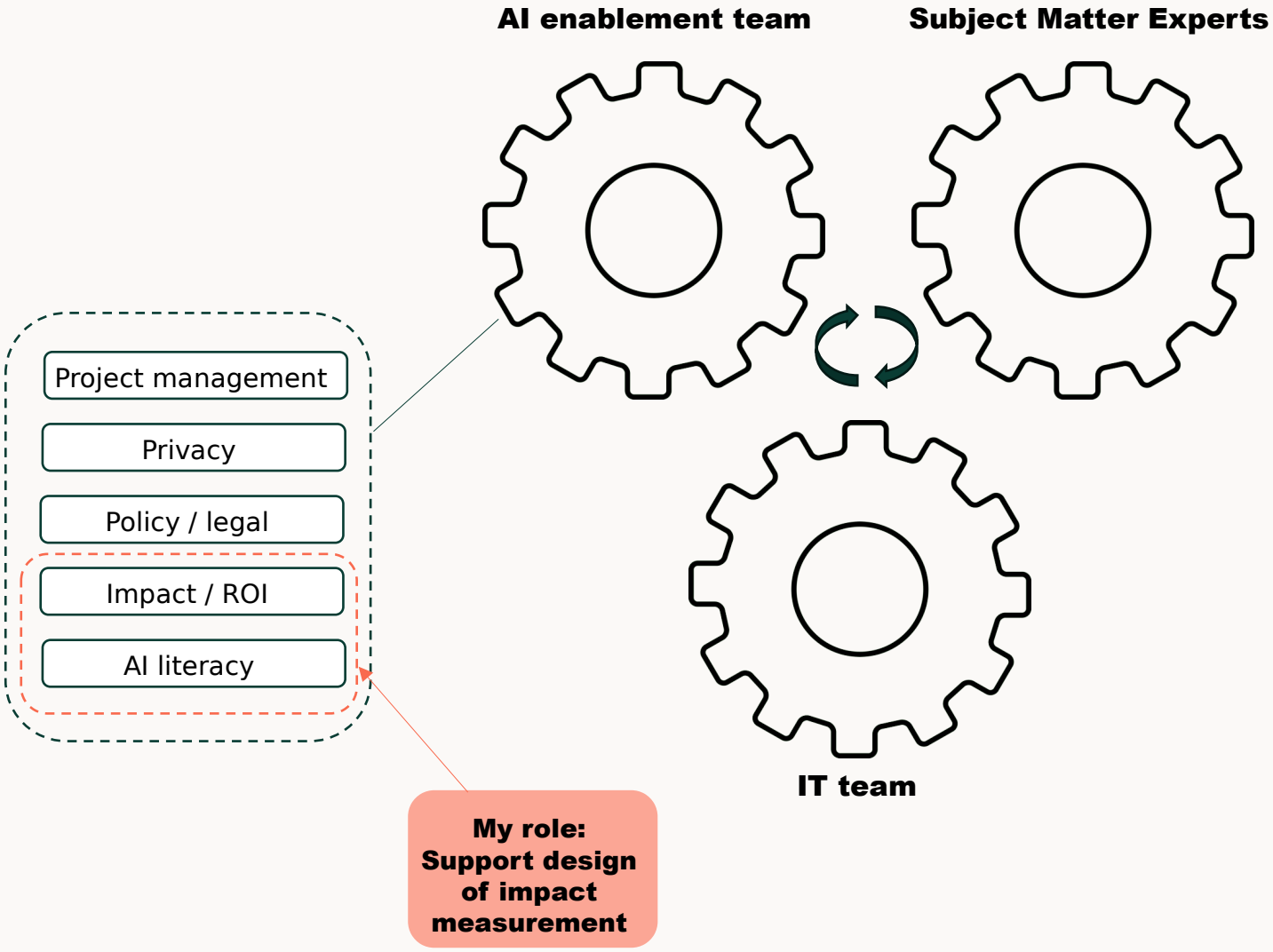
Introducing Dr Cody Dodd - Relevant work

London School of Economics (LSE): Ongoing research and collaboration on negotiated governance - how different algorithmic artefacts reshape control, accountability, and institutional design.

- Management Department
- School of Public Policy
- Methodology Institute
- Digital Skills Lab
- Data Science Institute

- **Public sector practice:** Directly support the governments and NGOs across Canada and the UK in deploying AI and automation through governance-first, privacy-aware architectures.
  - Institute for Citizen Centred Services
  - Federal (Service Canada), provincial/regional, municipal governments in Canada and UK
  - Canadian Research Insights Council

Introducing Dr. Cody Dodd - My typical role



Introducing Dr Cody Dodd - Relevant work

I also make video games! My AI work is respected in the field of entertainment as well :)

https://hermertiawars.com

Me presenting the upcoming Hermertia Wars to the G.R.I.D. Playtest Conference 2026, where we placed 5th out of 30th in ‘peoples choice’.

Day 1 - Foundations: What AI Is, Why Python Matters, and How to Run Code

Learning Goals:

- Participants can define the major types of AI

- Participants can run code in Codespaces

- Participants understand imports, pip, requirements.txt, and virtual environments

- Participants run a deterministic “hello world”

- Participants experience their first deterministic vs generative contrast: Compare LLMs to Python at predicting diabetes

Defining AI

What AI means today: The AI Effect

AI Effect – as championed by Pamela McCorduck

- The definition evolves over time as our understanding of older technologies becomes too common or ubiquitious.

“once upon a time, regression was called AI”

Are all LLM’s AI? Are all AI LLMs?

What AI means today: Copying human intelligence

1. Perception
- Vision (detecting objects, faces, scenes)
- Audio (speech recognition, voice-to-text)
- Touch / sensor interpretation

This course: computer vision, speech models, multimodal LLMs

2. Reasoning & Inference
- Logic, rules, decision trees
- Expert systems
- Early symbolic AI

This course: deterministic systems, classical ML, regression, clustering

3. Language & Communication
- Understanding text
- Generating text
- Translation

This course: transformers, GPTs, Claude, LLM

4. Learning
- Pattern recognition
- Prediction
- Adaptation

This course: supervised ML, reinforcement learning, fine-tuning

5. Action & Planning
- Sequencing steps
- Tool use
- Goal-directed behavior
- This course: agents, skills, workflow orchestration

Modern AI blends all five — but each family behaves differently.

What AI means today: Three families

AI is not one thing. It's three overlapping families:

1) **Deterministic AI**

2) Rules, thresholds, similarity, regression → predictable, explainable, reliable → the foundation of validation-first AI

3) **Machine Learning**

4) Models that learn patterns from data → classification, clustering, drivers analysis

5) **Generative AI (LLMs and GPTs)**

6) Transformers that predict the next token → powerful narrators, fragile decision-makers → great at language, weak at math and logic

The magic happens when you combine them.

Text AI?

Converts words to ‘tokens’, and tokens to ‘patterns’. Here is a simplistic example.

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0


```
"keyword_centroid": [
  -0.04863212630152702,
  0.14275532960891724,
  0.11660119891166687,
  -0.0429142527282238,
  0.0415881983935833,
  -0.3858243525028229,
  -0.2691216766834259,
  0.018577784299850464,
  0.03552545979619026,
  0.09642422944307327,
  0.12433751672506332,
  -0.47696685791015625,
  -0.07474993169307709,
  -0.2629881501197815,
  -0.16368107497692108,
  -0.23501090705394745,
  0.3147209882736206,
  -0.07454950362443924,
  0.35957419872283936,
  -0.11812958866357803,
  0.07812701165676117,
```

Why Python?

Python is the backbone of modern AI because it is:

- **Deterministic** — same input → same output

- **Auditable** — every step is inspectable

- **Reproducible** — version-controlled, stable

- **Extensible** — ML, data, APIs, agents

- The language LLMs call when they need real computation

- But also: the second most widely used language (after javascript) with a massive library of packages (see pip)

Python for our workshop is not here to make you a software engineer. It’s here to give you control.

Codespace

- Everyone launches a Codespace

- Those who want to use local machines can

- Quick tour: **file tree**, **terminal**, **Python version**, **requirements.txt**

How to start a Codespace

Go to the GitHub repo https://github.com/codydodd/ai-code-camp

Click “Code” → “Open with Codespaces”

GitHub launches a cloud-based VS Code environment

All packages install automatically from requirements.txt

You can run Python immediately — no setup required

Codespace versus self-hosting: when and why?

Why Codespaces?

- Identical environment for everyone
- No dependency conflicts
- No local installation issues
- No need to configure Python paths
- Perfect for short courses and workshops

Limitations

- Token costs if using LLM APIs
- Cloud compute limits (timeouts, memory)
- No GPU (fine for our course)
- Offline work requires local setup

When to use local Python instead

- When running long scripts
- When using large datasets
- When avoiding token costs
- When building something beyond a ‘pilot’.

Exercise: Hello world

- What is Hello World?

- Run a simple Python script

- Import a library

- Install a package

- Explain virtual environments (lightly)

requirements.txt

What is requirements.txt?
A list of all the Python packages your project needs.

Code

```
pandas==2.2.1
scikit-learn==1.4.0
tensorflow==2.15.0
```

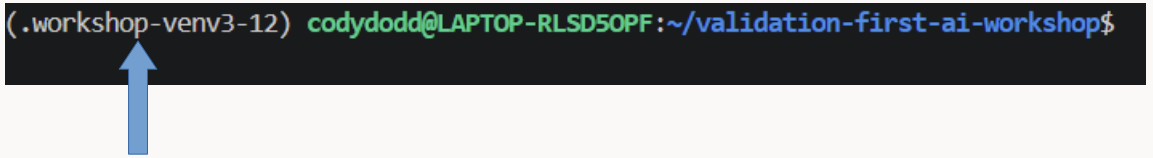
Why it matters
Anyone can recreate your environment
Codespaces installs everything automatically
Prevents “it works on my machine but not others” problems

Virtual Environments

You don’t need to master this today — but you will need it if you pursue more AI development.

What is a Virtual Environment?
A clean, isolated folder that contains:
- your Python version
- your packages
- your dependencies

Why it matters
- Keeps projects separate
- Prevents version conflicts
- Ensures reproducibility
- Keeps your computer clean!



This means my ‘terminal’ has an environment set up and turned on. Anything I install will go into here and not to my computer.

Deterministic versus Generative logic

Deterministic systems	Generative systems
Predictable	Non-deterministic
Explainable	Opaque
Reproducible	Variable
Auditable	Hard to trace
Grounded in math	Grounded in text patterns
Good for decisions	Good for narrative

Both can be ‘orchestrators’, but we will learn over this code camp the difference between deterministic orchestration and generative orchestration.

Exercise: Predicting diabetes

Steps:

- Participants run a deterministic script

- Participants ask Copilot/Claude to predict

- Participants compare outputs

- Participants observe drift, pollution, inconsistency

Exercise: Predicting diabetes

This activity introduces deterministic ML using the Pima Indians Diabetes dataset.

What to observe:

- Deterministic ML produces stable, reproducible predictions.
- Compare Python results to an LLM asked to predict diabetes without labels.
- Then compare again when the LLM is given labeled data.
- Try the same with the larger bank dataset.

Notice that some LLMs (e.g., Claude) get close only because they run Python tools, not because they “understand” the data.

Notice the ‘pollution’ given this is a very publicly available dataset.

This demonstrates the difference between simulation and computation.

Exercise: Detecting cat faces

- Introduction to bias

Introduction to the competition on Day 4

- All teams to split into groups (no more than 4 participants per group please)

- Develop an AI application, idea, or exploration

- Can be code, but does not need to be. You can also ‘reflect’ on AI use and impacts. <-- sometimes these win the ‘mindfulness’ award!

- Can get started on Day 1, but will get dedicated time to finish on Day 4

- Previous year winners:

- Predict disasters and food shortages for a humanitarian platform
- Investigate the state of LLMs in Kazakh
- Summarize politicians by their social media content
- Fashion designer using computer vision

Introduction to the competition on Day 4

“What Makes a Good AI Project?”

- Small scope

- Clear data

- Good understanding of where the deterministic parts are, and the generative parts

- Mindful of risk

- Something we can finish in 1.5 days

YES use Claude to build your code.

Day 2 - Ensemble Thinking, LLM Limits, and Practical ML

Learning Goals:

- Participants understand ensemble AI

- Participants see LLMs fail at structured tasks

- Participants understand hallucinations, sycophancy, pollution, non-determinism

- Participants run the thematic analysis pipeline

- Participants begin thinking about project ideas

AI systems can fail in predictable ways:

- Bias amplification

- Privacy leakage

- Hallucinations

- Sycophancy

- Training data pollution

- Non-determinism

- Overconfidence

- Opaque reasoning

Your job is not to avoid risk. It’s to design systems that manage it.

AI systems can fail in predictable ways:

- COMPAS case

- Misgendering

- Awful history of abusive ‘training’

https://time.com/6247678/openai-chatgpt-kenya-workers/

MENU ≡ TIME SUBSCRIBE

BUSINESS + TECHNOLOGY +

Exclusive: OpenAI Used Kenyan Workers on Less Than $2 Per Hour to Make ChatGPT Less Toxic

ADD TIME ON GOOGLE

by Billy Perrigo
CORRESPONDENT

JAN 18, 2023 12:00 PM GMT

This image was generated by OpenAI's image-generation software, Dall-E 2. The prompt was: "A seemingly endless view of African workers at desks in front of computer screens in a printmaking style." TIME does not typically use AI-generated art to illustrate its stories, but chose to in this instance in order to draw attention to the power of OpenAI's technology and shed light on the labor that makes it possible. *Image generated by Dall-E 2/OpenAI*

The risks we know

Poor AI governance = Democratic failure

- **95% of AI investments fail to deliver** (MIT Labs + HBR)
- **41% of workers encounter workslop monthly** (fake or empty outputs), trust lost for public institutions.
- **Only 3 out of 45 real sources** in a major study - KPMG
  - https://app.gptzero.me/documents/45f1e69a-991c-44d8-ad05-48036fba5085?scanType=bibliography_scan

Non-deterministic.

- Anyone who tells you a GPT can be trained to avoid mistakes, or that it just needs 'better prompting', is misunderstanding their architecture.
  - "It ignored one of my requests no matter how many times I asked it not to"
  - "It erased data it shouldn't have"
  - "It summarized only parts"
  - "It made that one up"

"Context is a Goldilocks zone. Too little, it guesses, too much, it gets confused or takes shortcuts"

GPT benefits

Reformatting

Combine, summarize, tighten, change formats, add documentation.

Faster starts

Remove that slow start / blank page.
Create quick drafts.

Makes us more creative.

‘Have you thought of?’

Quick pass to find spelling errors, pointers, change language style.

Faster coding

Generate scripts, build quick pilots, describe other code.

GPT challenges

- Difficult to capture ROI of GenAI as a productivity tool.
- Weak signals get amplified (with bias implications)
- Ungoverned systems hide accountability
- Algorithmic pipelines decide who and what gets heard
- Retrieval tends to skips core sections
- Not to mention data sovereignty, prohibitive costs, environmental impact

Regulatory tension

These regulatory contexts:

- GDPR; EU AI Act; Canada's AIDA (Bill C-27) and "Guiding Principles for the Use of AI in Government"; UK's ICO guidance and core AI principles

- expect AI to be:

1) **Explainable** (You can show why an output happened)

2) **Traceable** (You can show what the system did, step-by-step)

3) **Human-oversighted** (A person can review, correct, and override)

4) **Purpose-limited & data-minimal** (Only the necessary data, used for a clear purpose)

Regulations assume systems behave predictably — GenAI doesn't.

Comparison

	LLMs	Determinstic ML
Determinism	Non-deterministic — same prompt can yield different outputs	Predictable — same inputs always produce the same outputs
Grounding	No grounding — outputs are generated from patterns in text, not computation	Grounded — outputs come from explicit mathematical models
Reproducibility	Low — temperature, sampling, context drift, and hidden heuristics change results	High — fixed algorithms, fixed seeds, fixed data
Auditability	Weak — internal reasoning is opaque and cannot be inspected	Strong — every step is inspectable, traceable, and explainable
Error Behavior	Silent failure — confident but wrong answers	Transparent failure — errors are measurable and debuggable
Data Use	Trained on broad text; not tied to your dataset unless explicitly given	Trained only on your dataset; behavior is fully controlled

Exercise: LLMs versus Python - Regression / Drivers analysis

Steps:

- Participants run a Random Forest script

- Participants ask LLMs for drivers

- Participants compare all LLM answers

- Participants see reproducibility vs drift and understand the presence of hidden sampling parameters

Exercise: LLMs versus Python - Regression / Drivers analysis

Start with the Wine Quality dataset, then the Airline Satisfaction dataset.

What to observe:
- Random Forests provide ranked feature importances.
- Compare these to LLM-generated “drivers.”

Some LLMs get close only when they execute real Python, not when they guess. Others hallucinate drivers entirely. We may also get zero overlap, different results from everyone. This is due to hidden parameter logic.

Why LLMs Behave This Way

Why LLMs Behave This Way (Architecture Explanation) - Hallucinations

LLMs are next-token predictors, not reasoning engines. Their architecture creates these limitations:

- **They generate text, not truth.**
  Every output is a probability distribution over words, not a computation.

- **They have no internal concept of "correctness."**
  They optimize for plausibility, not accuracy.

- **They do not execute algorithms unless explicitly connected to tools.**
  Without Python or deterministic components, they simulate math instead of doing math.

- **Their outputs depend on hidden sampling parameters.**
  Temperature, top-p, and context windows introduce randomness.

- **They cannot trace or justify their internal steps.**
- The transformer architecture does not expose intermediate reasoning.

- **No stable memory**
  Memory and context is ever-changing. They also 'summarize' data instead of parsing all of it, and do a lot of 'guessing'

This is why LLMs are powerful narrators, but unreliable predictors

Why LLMs Behave This Way

Why Sycophancy Happens

- Reinforcement from human feedback
- Optimization for agreement
- Lack of epistemic confidence
- Social mimicry baked into training

Why Pollution Happens

- Public datasets
- Web-scale training
- Retrieval contamination
- Overfitting to common benchmarks

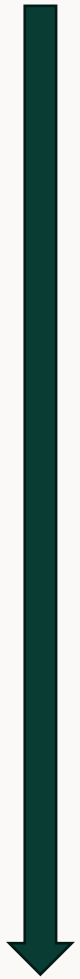
Deterministic ML models:

- Compute, they don’t guess
- Use fixed mathematical functions
- Produce the same result every time (if configured to)
- Are fully inspectable (coefficients, feature importances, SHAP, etc.)
- Can be version-locked, audited, and certified
- Are required for regulated environments (finance, government, health)

Ensemble AI - Combining deterministic and non-deterministic logic

- Case study: Text analysis in the public sector

Validation-first AI



Start with deterministic systems:

- Classic tools like z-scores, crosstabs, regression still play a role. These can predictably cluster data, detect outliers, and build grounded truth using rules-based logic.
- For text analysis: “Voice of Citizen”:
  - Keyword-based matching – Most trusted, least flexible
  - Include lemmas and other language rules – Still predictable, more flexible
  - Matches based on token similarity – Less predictable, more flexible
  - Machine-learning based on human tagging – More predictable, more flexible, but requires training.

Once truths are established, turn generative AI and agents:

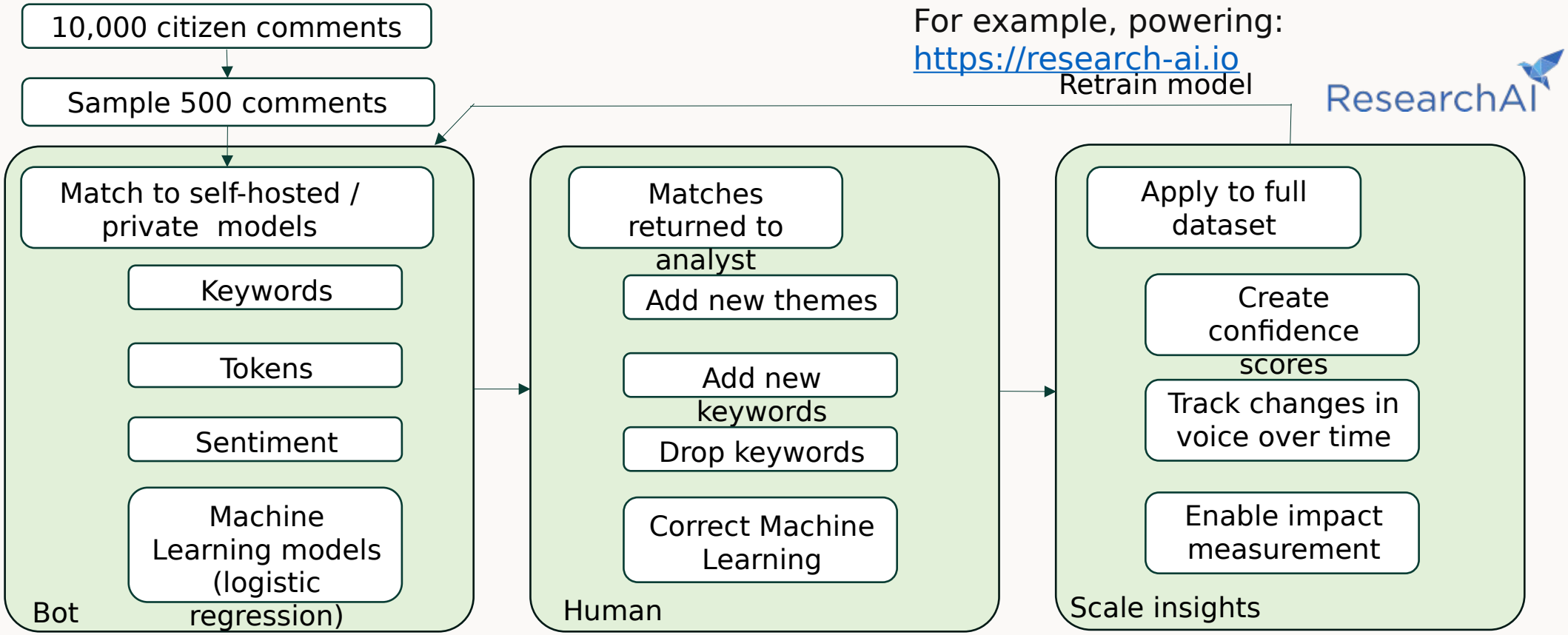
- Summarization on top of grounded truth.
- Treat as suggestive, not authoritative.

Ensure steps produce metadata:

- Track what algorithms did what when.

Methodology: Validation-first text analysis AI

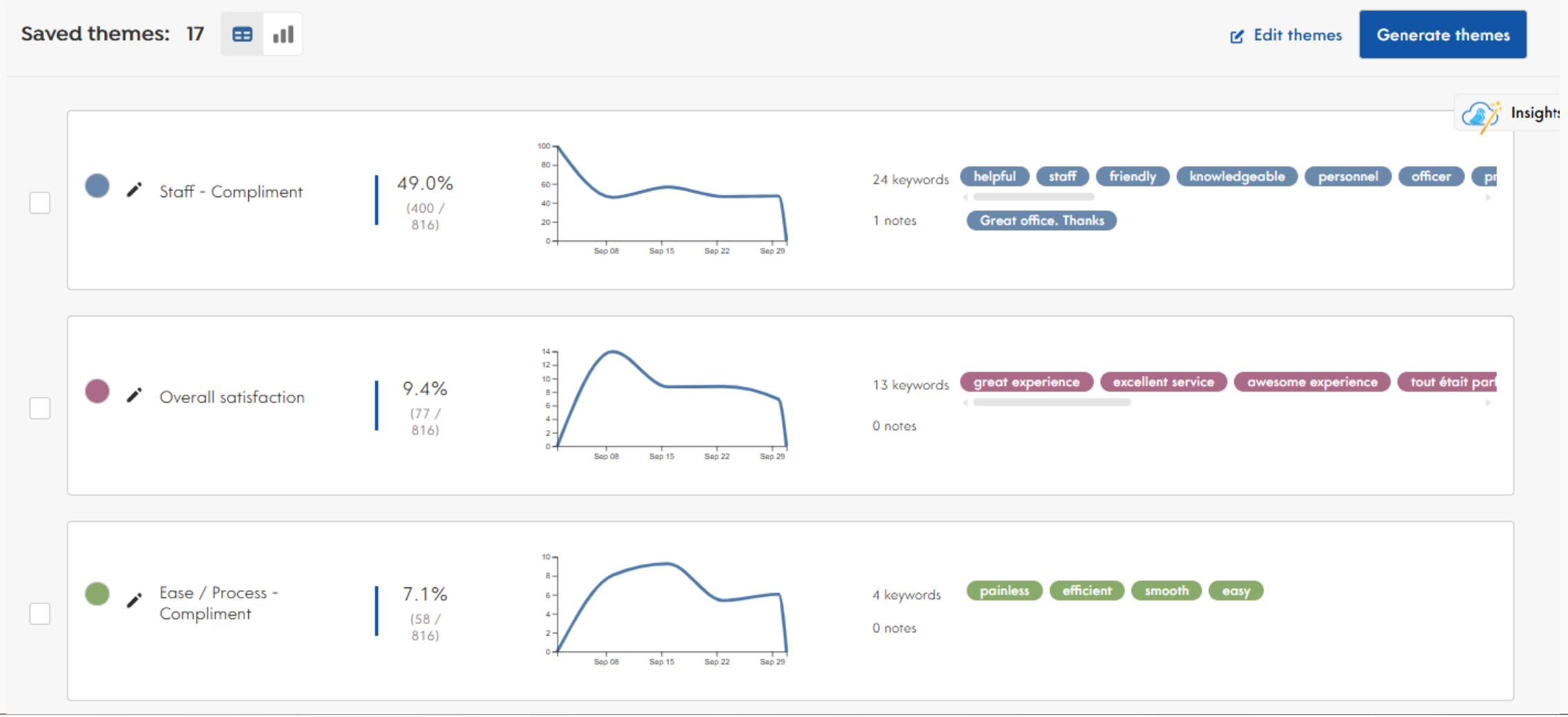
Canadian Federal department, Yukon Government, Peel region, and dozens of other governments: capturing voice and building trust using governed architecture. **Thousands of citizen comments analyzed a month, with transparent, reproducible tags of exactly what is being said**. This is an example of validation-first AI.



Methodology: Validation-first text analysis AI

Example of Validation-first-AI supporting theme tracked over time

ResearchAI



Methodology: Validation-first text analysis AI

Who is using this:

- Yukon Government now tracks emergency services in real-time, redistributing resources based on citizen voice.
- Canada's largest service department validates web and operational data using voice of client.
- Parliamentary committees are tracking changes in parliamentary priorities from transcripts.

Examples of how to drive citizen engagement and trust:

- When a new program is launched, set target for change in themes
  - "A consistent top complaint is difficult authentication for the digital service (10% of comments each month), the new log-in feature should decrease this to 5%."

- Automatic red-flag reports when certain topics reach certain thresholds
  - "Complaints about fraud should never exceed 2% of all comments"

Methodology: Validation-first text analysis AI

Ingestion and cleaning (deterministic)

Similarity and clustering (deterministic)

Human validation (governance)

Machine learning from human actions (deterministic, negotiated logic)

GenAI summarization and narrative building (constrained)

Transparent reporting (reliable insights retraceable back to citizen comments)

Exercise: Predicting Themes (Dictionary vs ML vs LLM)

- **Steps:**
- Review and run dictionary-based themes
- Review and run naive ML to find themes
- Review and run weighted ML using Research AI
- Get LLMs to tag
- Compare accuracy
- Discuss reliability

Exercise: Predicting Themes (Dictionary vs ML vs LLM)

This activity uses a dataset of ~900 human-tagged comments.

What to observe:
- Run a dictionary-based topic model using custom_topic_model-online_application.xlsx.
- Then run a machine learning classifier trained on human tags.
- Compare both to a pre-tagged dataset from a more complex ML model.
- Finally, ask an LLM to replicate the tagging.

We see:
- Deterministic dictionary methods are transparent but brittle.
- Generic ML models are not necessarily as accurate as keywords, without extensive coding.
- More advanced / managed ML models can get remarkably accurate with limited tagging.
- Without python, LLMs are inconsistent, often drifting or hallucinating categories.

This demonstrates the limits of generative systems for structured classification.

Claude is trained to turn to predictable, deterministic tools. It is using LLMs to ‘orchestrate’ but not ‘compute’.

Binary classification predictions for CSV data

experience with government online social security applications.
You will:
1. Read all responses I provide.
2. Identify a set of discrete, non-overlapping, service-experience themes.
3. Create a final theme list that:
- Is concise (15–40 themes)
- Uses short, clear labels
Show more

Jun 14

Verifying zero-theme rows and scrutinizing high-frequency theme precision

Now adding the Complaint pattern, Application Status "issued or not", and verifying the duplicate `key for...log` line doesn't remain:

Edited 2 files, ran a command

Check for duplicate key for log pattern

Script

Classify v3 · PY

Copy

```python
import pandas as pd
import re
import unicodedata

df = pd.read_excel('/mnt/user-data/uploads/text_analysis_online_social_security_application_no-results.xlsx')

def normalize(text):
    text = str(text)
    text = text.replace('’', "'").replace('‘', "'")
    nfkd = unicodedata.normalize('NFKD', text)
    no_accent = ''.join(c for c in nfkd if not unicodedata.combining(c))
    t = no_accent.lower()
    return t

def augment(t):
    """Add canonical tokens for common spelled-out / misspelled benefit names
    to make downstream pattern matching robust, without destroying original text."""
    extra = []
    gis_patterns = [
        r'guarante?e?d?\s+income\s+suppl?ement', r'guaranteed income supplement',
```

Cloud Costs, Token Costs, and Practical Constraints

Token Costs
- LLMs charge per input + output token
- Thematic analysis can require hundreds of thousands of tokens
- Claude's Python retries can multiply cost
- Costs scale with:
  - dataset size
  - prompt length
  - number of retries
  - number of participants

Cloud Compute Costs
- Codespaces have time limits
- LLM calls can time out
- Heavy tasks (CV, audio) may be slow
- Cloud environments can throttle usage

Why This Matters
- Deterministic ML is free once installed
- LLMs are variable cost
- Agents can be expensive if poorly designed
- Real organizations must budget for this
- This is why validation-first AI reduces cost and risk.

Not to mention environmental, social, and ethical costs

Play with code

Other examples to play with:

- Voice-to-text

- Computer vision

- Sentiment analysis

- Participants to explore packages

Day 3 - Agents, Orchestration, and Building Real Systems

Learning Goals:

- Participants understand agentic logic

- Participants understand the difference between LLM orchestration and Python orchestration

- Participants see Claude Agents in action

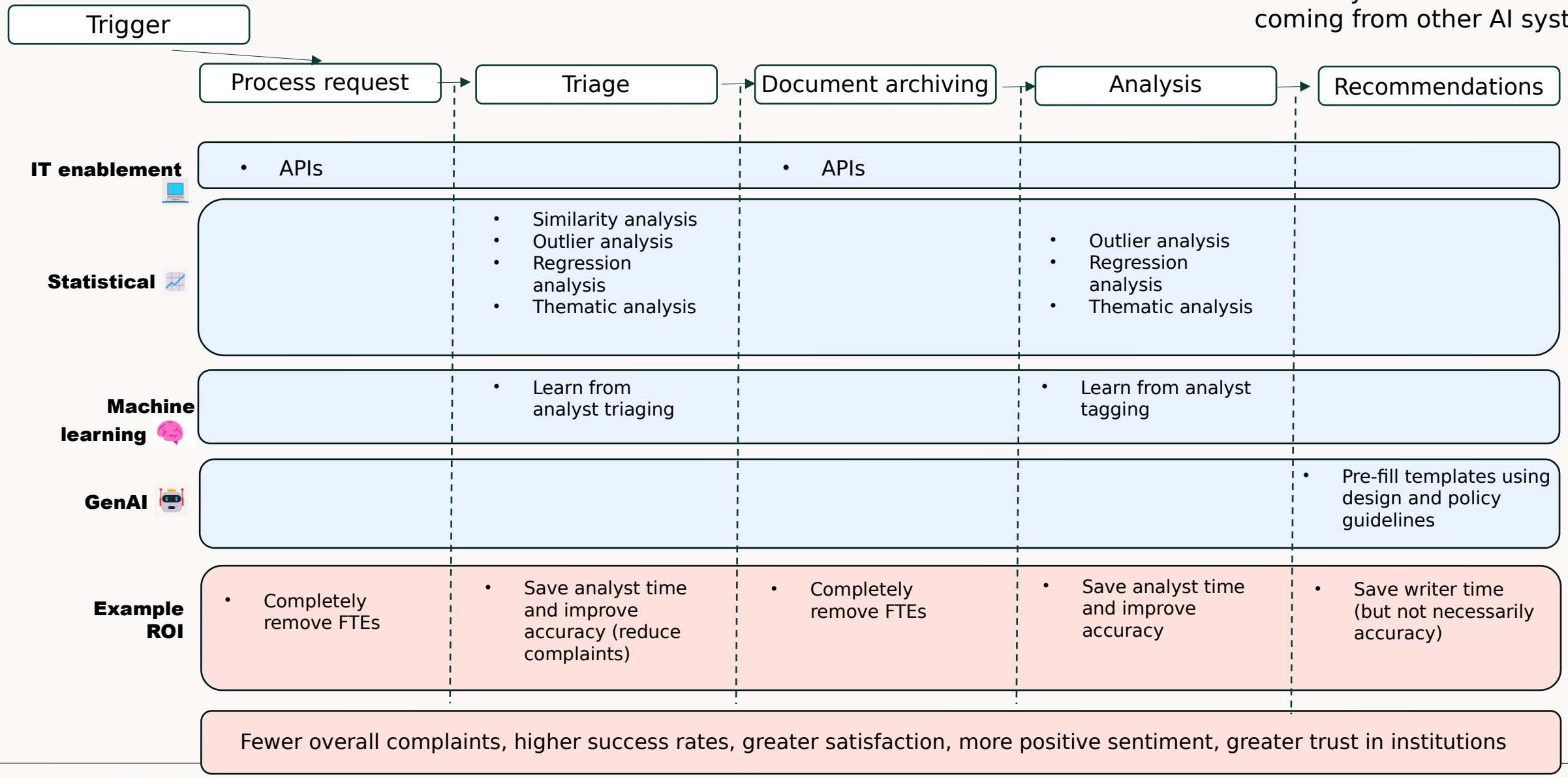
- Participants begin building their project

Ensemble AI - Combining deterministic and non-deterministic logic

- Case study: AI Enablement and workflow automation in the public sector

Example: Triage automation solutions

Notice: very little GenAI: most value coming from other AI systems



AI enablement elements

	What It Does	Strengths	Weaknesses	When to Use
IT Enablement	APIs, integrations, workflow triggers, routing, data access	Reliable, scalable, zero hallucinations	Requires engineering	Always first — foundation for automation
Statistical / Deterministic	Rules, thresholds, similarity, z-scores, crosstabs	Predictable, auditable, explainable	Less flexible	Intake, triage, validation, early classification
Machine Learning	Logistic regression, Random Forests, clustering	Accurate, reproducible, learns from humans	Needs training data	Tagging, prioritization, risk scoring
Generative AI	Drafting, summarizing, narrative building	Fast, expressive, creative	Non-deterministic, not auditable	Often last — after truth is established

Output: Automation use-cases

	AI Service Capability (Primary Problem)	What this is	AI solution
1	**Request Intake & Triage (single door)**	(Channel-agnostic but not channel optional) Multi-channel, compliant request intake with early completeness validation, request classification (simple/complex), risk-based triage, categorisation, prioritisation, and controlled nudging toward preferred digital channels.	Deterministic workflows to detect likely categories for triaging and auto-fill forms. No GenAI or agents needed
2	**Tasking, Tracking & Routing (orchestration)**	End-to-end orchestration of work items across systems, including assignment, status visibility, escalation, exception handling, and audit-ready traceability.	Deterministic workflows for detecting likely tasks and routing. No GenAI needed, but agents could support.
3	**Drafting & Content Creation**	First-draft creation using authoritative content sources and templates, supported by pre-draft redaction where required, with mandatory human validation and version control	GenAI to consume templates to provide suggestive responses. Humans must finish the draft. Deterministic 'similarity' analysis to provide suggestions.
4	**Governance & Committee Support**	End-to-end governance enablement including intake, records, metadata, decision traceability, and scalable onboarding across formal and informal committees.	CMS and data access is the main challenge, not automation. The solution explored better utilizing existing CMS systems.
5	**Transcription & Summarisation**	Secure meeting capture, assisted summarisation, and records-of-decision creation, with human validation, language handling, and retention controls.	Summarization and transcription speeds up archiving but French vs. English translations still struggle.
6	**Information Storage, Access & Retrieval**	Discovery, retrieval, preparation, and secure delivery of authoritative information and records across repositories, including digitisation, deduplication, sensitivity handling, and controlled external access.	Data access remains a main challenge (similar to area 4). API development can produce more savings.

What Agents Actually Are

- Not magic

- Not autonomous

- Not “AI that thinks”

- They are workflow engines with LLM glue

Showcasing Claude Agents

What it is:

When you use claude on claude.ai you are using the ‘chat’ feature.

Claude Agent (or Claude Code) is a different layer that has ‘tools’ to help you build workflows.

- Read / write files
- Run terminal commands
- Search codebases
- Call external APIs
- Spin up sub-agents

Claude agent decides itself what order to use.

https://www.natecue.com/en/learn/ai/claude-agent-and-skills/

Showcasing Claude Skills

What it is:

An ‘md’ file containing name and descriptions, step by step instructions, outputs and formats, and other rules.

Think of it as a refined prompt that you can save and re-use later.

https://www.natecue.com/en/learn/ai/claude-agent-and-skills/

Showcasing Claude triggers

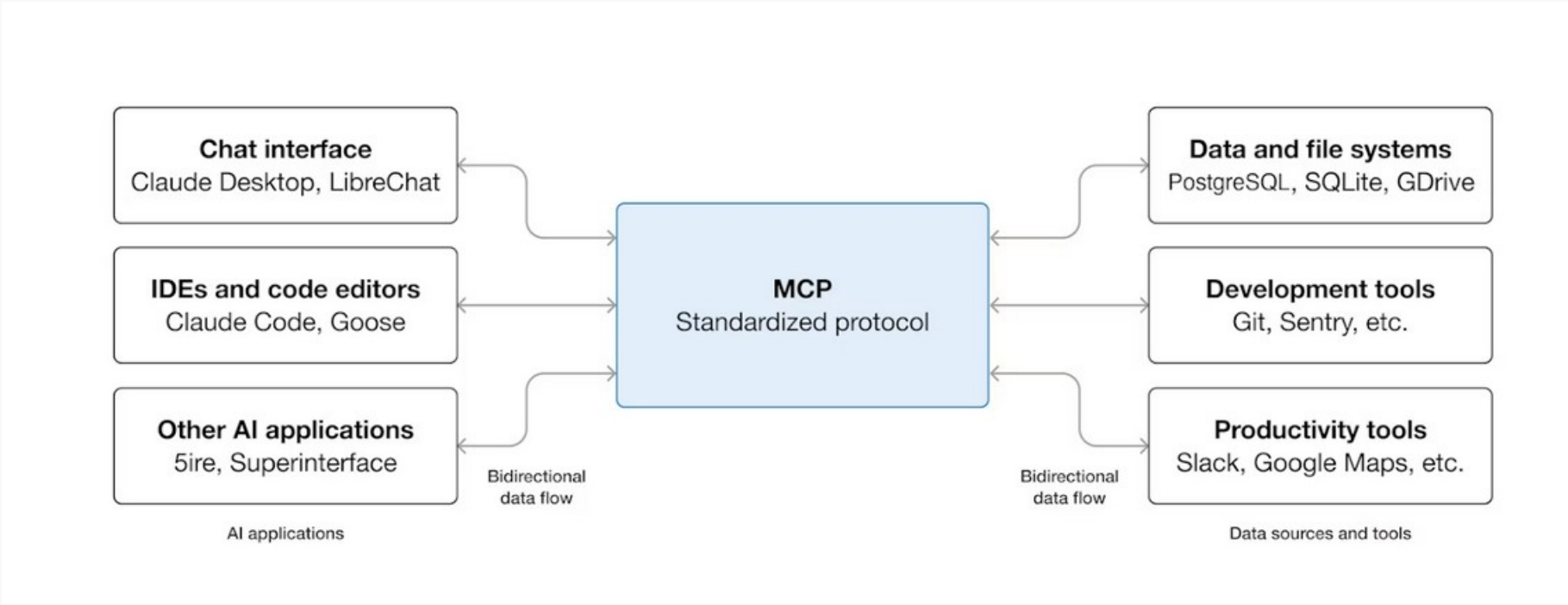
What it is:

- actions that initiate Claude Agents, such as using ‘/’, when certain information appears in prompts, or when something is @mentioned

https://www.natecue.com/en/learn/ai/claude-agent-and-skills/

Standardizing connections

Claude (and other AI applications) are able to more seamlessly connect data and systems using MCP's



https://claude.com/blog/skills-explained

Claude Chat — “The Conversational Brain”

Free-form conversation

Best for: summarizing, drafting, brainstorming, explaining

Behaves like a classic LLM (predicts text)

Not deterministic

Doesn’t run code

Doesn’t enforce structure unless you add a Skill

Use when: You want ideas, explanations, rewriting, or quick Q&A.

Claude Cowork — “The AI Research Assistant”

A guided workspace where Claude helps you step-by-step

Can analyze files, documents, datasets

Often includes tool use (Python sandbox, file reading)

More structured than Chat, less structured than Code

Great for: analysis, debugging, exploring datasets, multi-step tasks

Use when: You want Claude to work with you on a task, not just talk about it.

Claude Code — “The Coding Workbench”

A dedicated coding environment

Claude writes, edits, explains, and debugs code

Includes a Python execution sandbox

Can read/write files, run scripts, show logs

Deterministic execution (Python), generative planning (LLM)

Use when: You want Claude to execute code, run workflows, or perform deterministic analysis.

Projects — "Your AI Workspaces"

A container for files, datasets, notes, and conversations

Lets you organize multi-day or multi-file work

Claude remembers context within a project

Perfect for your Day 4 student projects

Use when: You want a persistent workspace with files + conversations.

Scheduled — "Run Something Later"

Automate tasks on a schedule

Example: "Every Monday, summarize this dataset"

Useful for ongoing monitoring or repeated analysis

Use when: You want recurring automated tasks.

Skills — "Reusable Mini-Workflows"

Custom instructions you can attach to Claude Chat

Enforce structure → reduce drift

Great for repeatable tasks (e.g., drivers analysis, summarization templates)

Deterministic behavior, even though the model is generative

Use when: You want Claude to follow strict steps every time.

Dispatch — "Trigger a Workflow from Outside"

Lets external systems trigger Claude workflows

Think: API calls, webhooks, integrations

More advanced, but great for enterprise automation

Use when: You want Claude to react to external events.

Artefacts — "Generated Files & Outputs"

Code files, reports, diagrams, datasets Claude creates

Stored automatically

You can open, edit, download, or reuse them

Great for: notebooks, scripts, CSVs, diagrams

Use when: Claude produces something you want to keep or reuse.

The power of typical LLMs

Copilot followed our instruction:
"You are a binary classifier."

But what it actually did was:

- read each row as text
- apply loose, heuristic pattern-matching
- guess based on correlations it has seen in training
- output 0 or 1 based on those heuristics

It did not:
- train a model
- optimize weights
- evaluate accuracy

It simply simulated the behaviour of a classifier.

This is why copilot can feel like it's doing ML — but it isn't.

Why the manual deterministic model may be better

- it actually trains
- it optimizes weights
- it learns nonlinear boundaries
- it is consistent
- it is reproducible

The power of Claude

	Typical LLMs	Claude
Tool Use	Rarely uses tools reliably	Strong, consistent tool-calling (Python, retrieval, code execution)
Numeric Reasoning	Simulates math → errors	Delegates math to Python → grounded results
Following Instructions	Drifts, shortcuts, ignores constraints	More disciplined, more literal
Hallucination Control	High	Lower (but not zero)
Reproducibility	Low	Higher, but still non-deterministic

Claude is still a generative model, but its architecture and training emphasize:

- Tool orchestration over simulation
- Delegation over guessing
- Grounding over plausibility
- Refusal over hallucination

This makes Claude behave closer to a validation-first system — especially when it chooses to run Python instead of predicting.

Even with strong tool-use:

- It can still ignore a tool call
- It can still hallucinate when uncertain
- It can still drift with long context
- It can still produce different outputs across runs
- It cannot be audited internally
- It cannot be version-locked by the user

Claude is the best of the generative models for structured tasks — but it is still a probabilistic narrator, not a deterministic engine.

Beyond LLMs – the power of Agents and Skills

Agents and skills introduce deterministic structure around an LLM. They reduce drift by giving the model tools and rules instead of letting it improvise

	LLM Alone	Agent / Skill Layer
Task Execution	Simulates steps	Executes real steps (Python, APIs, workflows)
Consistency	Variable	Deterministic nodes + repeatable logic
Memory of Process	Weak	Explicit plans, state, and checkpoints
Error Handling	Unpredictable	Rules, retries, fallbacks
Data Access	Limited to prompt	API calls, databases, validated inputs
Auditability	Opaque	Logs, traces, reproducible runs

Agents don’t make LLMs deterministic — but they wrap them in deterministic components:

- Python nodes
- API calls
- Validation steps
- Rule-based checks
- Workflow engines

This creates a hybrid system: LLM for language → deterministic tools for everything else.

Agentic Orchestration vs Python Orchestration

- Visual programming vs real programming

- When to use each

- Why both matter

- How they complement each other

Dimension	Agentic Orchestration (Claude Agents)	Python Orchestration
How it works	LLM chooses which tools to call	Developer writes explicit logic
Control	Medium — LLM decides steps	High — every step is defined
Determinism	Low–medium	High
Complexity	Easy to start, hard to debug	Harder to start, easy to debug
Error Handling	LLM guesses unless rules added	Full control (try/except, logging)
Data Access	Through tools the LLM chooses	Through APIs you explicitly call
Auditability	Limited	Full logs, reproducible
Best For	Prototyping, drafting, light workflows	Production systems, reliability
Worst For	High-risk decisions	Rapid experimentation

Will organizations ever need Python again?

Why Python will always matter

- APIs require deterministic logic
- Data pipelines require reproducibility
- ML models require training code
- Agents require tools to orchestrate
- Governance requires auditability
- LLMs cannot replace computation
- LLMs cannot replace workflow engines

Where Python fits

- The backbone of IT enablement
- The foundation of deterministic logic
- The engine behind ML models
- The tools that agents call
- The glue that connects systems

Where agents fit

- Light orchestration
- Drafting
- Summaries
- Human-in-the-loop workflows
- Agents don’t replace Python — they wrap it.

Concluding thoughts

Take-aways for AI and democratic insights

AI works best as a system, not a tool:

- Governance privacy, policy, IT, and SME's must co-own it
- Be sure to have AI literate voices at the table (who can tell the difference between deterministic and generative systems, who understand sources of bias and hallucinations, etc)

Deterministic automation delivers the biggest ROI:

- Triage, routing, validation, and intake outperform GenAI for reliability, ease-of-adoption, and impact.

GenAI is powerful but fragile:

- Best used for drafting, summarizing, and templated content after validation.
- Can find compelling narratives, but not strong at maintaining good sources.

Validation-first can produce reliable measures of citizen voice:

- Self-hosted models that rely on deterministic logic (avoiding hallucinations and fabricated comments) help us trust text analysis, and in turn, drive policy and service improvement.

Governance-first teams outperform tech-first teams:

- because they understand risk, accountability, and citizen-impact

Transparency builds trust:

- When citizens see how their input was analyzed, trust increases measurably.

AI Automation enablement

Different algorithms do different things

- Be confident enough to ask when should we use a deterministic process versus a generative one

- **Risk can be managed**
  - We don’t need a one size-fits-all solution. We can manage risks by building an ecosystem of different systems
- **Teams can be empowered**
  - Yes agents, but don't forget python.
  - Code-camps for non-coders + GenAI for code generation has led to teams finding programming superheroes they didn’t know they had

Copilot alone isn’t enough.

Advantages:

- Faster deployment of AI with real results
- ROI measured in down-stream savings and improved citizen trust and engagement

Take-aways for AI in better insights

AI that aligns with market research values looks like this:

- Predictable systems (deterministic steps you can explain and audit)
- Human-validated insights (voice interpreted with care, not hallucinated)
- Data-sovereign architectures (on-premise, transparent, privacy-first)
- GenAI used safely (as a narrator on top of validated truth, not a decision-maker), or if properly managed, as an orchestrator

Why it matters for democratic institutions:

- Clearer citizen voice → better policy signals
- Traceable pipelines → higher institutional trust
- Transparent methods → stronger legitimacy
- Reliable insights → foundations for deliberative democracy

Take-aways for AI in better insights

Other topics to explore (Optional):

1) What is GIT and why do we need it?

2) Tokenization under the hood

3) Other sources of bias

4) Self-hosting LLAMA

Future work and contacts

From bureaucracy to a more deliberative ballot:
Advancing voice of client from service delivery, policy, and parliamentary management to supporting parliamentarians, NGOs, and other democratic and community institutions to incorporate citizen voice in reliable, transparent ways. This could open the door for deliberative democracy.

Governing generative platforms:
Upcoming research on how generative systems re-write the platforms literature which once assumed stable cores. Cores are now dynamic, and once-dynamic periphery features are now becoming stable to act as ‘guard rails’ for unpredictable cores.

Let’s continue the conversation! If you’re working on governing AI, citizen insight, or automation at scale—and want to avoid black-box decision-making—I’d welcome the conversation.

Dr. Cody Dodd
Researcher, instructor & practitioner in AI governance

🔗 Linkedin: https://codydodd.ca
✉ Email: cody@codydodd.ca

Project kick-off

Participants to pitch ideas

Cody will help map the lanes:

- IT enablement

- Deterministic logic

- ML

- GenAI

- Agents